

宁波大学
NINGBO UNIVERSITY

实 验 报 告

[实验十 Linux 多任务编程实验]

姓名：	赵瑞杰
学号：	216001944
课程名称：	嵌入式系统
所在学院：	信息科学与工程学院
专业班级：	21 级计算机科学与技术 1 班
任课教师：	王翀、王艺睿

实验项目名称	**实验十 Linux 多任务编程实验**

一、实验目的与要求

1.1 实验目的

(1) 理解 Linux 系统中进程的概念以及进程管理的基本原理。

(2) 学习使用 Linux 系统工具查看和管理系统中的进程。

(3) 掌握 Linux 多任务编程的基本方法，理解进程间的关系和通信方式。

1.2 实验要求

(1) 实践编写守护进程程序，理解守护进程的编写步骤和特性。熟悉使用 ps 命令和/proc 文件系统来查看系统中的进程信息，包括进程 ID、状态、占用资源等。

(2) 编写程序使用 fork、exit 和 exec 系统调用实现多进程程序，理解这些调用在多进程编程中的作用。

(3) 学习进程间通信的基本方式，包括但不限于管道、消息队列、共享内存等，并尝试实现简单的进程间通信。

(4) 理解 Linux 守护进程的概念和特性，学习如何编写和部署一个简单的守护进程。

二、实验环境

(1) 华清远见开发环境。

(2) 实验平台为华清远见提供的虚拟机环境。

(3) 使用的操作系统是 Ubuntu 12.04 LTS 64-bit。

(4) 虚拟机工具软件为 VMware Player。

三、实验步骤、内容与结果

3.1 使用 ps 命令查看进程信息

在 shell 提示符下输入如下命令，并解释输出的结果：

```
$ ps
```

终端显示正在运行的对应进程：

```
linux@ubuntu64-vm:/proc$ ps
  PID TTY          TIME CMD
 2519 pts/0    00:00:00 bash
 3161 pts/0    00:00:00 ps
```

3.2 使用 proc 文件系统查看进程信息

认识 proc 文件系统的文件和目录：

```
$ cd /proc
$ ls
```

```
linux@ubuntu64-vm:/$ cd /proc
linux@ubuntu64-vm:/proc$ ls
1     1140  2061  2219  232   2519  4    770          fs            partitions
10    1141  208   2220  2320  2581  494  8            interrupts    sched_debug
1042  1161  2096  2223  2327  2582  495  860          iomem         schedstat
1049  1199  21    2229  233   2598  551  9            ioports       scsi
1057  12    2107  2232  2350  26    552  906          irq           self
1059  1228  2141  2234  2359  2610  577  952          kallsyms      slabinfo
1063  1293  2146  2237  2372  2616  585  acpi         kcore         softirqs
1081  13    2147  2238  2387  2621  586  asound       key-users     stat
1084  14    2152  2240  24    2630  59   buddyinfo    kmsg          swaps
1087  1402  2156  2246  2412  2665  6    bus          kpagecount    sys
1088  15    2159  2261  2414  2684  612  cgroups      kpageflags    sysrq-trigger
1092  16    2160  2263  2416  27    628  cmdline      latency_stats sysvipc
11    17    2164  2264  2419  2712  629  consoles     loadavg       timer_list
1108  1741  217   2267  2421  2961  631  cpuinfo      locks         timer_stats
1131  1770  2170  2272  2446  3     632  crypto       mdstat        tty
1132  18    2176  2277  2462  3052  639  devices      meminfo       uptime
1133  1803  2178  2290  2464  3058  646  diskstats    misc          version
1134  1848  2186  2292  2470  316   688  dma          modules       version_signature
1135  1860  2198  2297  2494  320   7    dri          mounts        vmallocinfo
1136  19    22    23    25    35    718  driver       mpt           vmstat
1137  1910  2203  2303  2508  37    729  execdomains  mtrr          zoneinfo
1138  2     2211  2314  2514  38    746  fb           net
1139  201   2218  2319  2518  39    762  filesystems  pagetypeinfo
```

通过 proc 文件系统查看系统当前进行状态：

```
$ cat /proc/self/status
```

```
linux@ubuntu64-vm: /proc
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 109 124 1000
VmPeak:     8904 kB
VmSize:     8904 kB
VmLck:         0 kB
VmPin:         0 kB
VmHWM:       368 kB
VmRSS:       368 kB
VmData:      176 kB
VmStk:       136 kB
VmExe:        44 kB
VmLib:      1928 kB
VmPTE:        40 kB
VmSwap:        0 kB
Threads:        1
SigQ:   0/11915
SigPnd: 0000000000000000
ShdPnd: 0000000000000000
SigBlk: 0000000000000000
SigIgn: 0000000000000000
SigCgt: 0000000000000000
CapInh: 0000000000000000
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapBnd: ffffffffffffffff
Cpus_allowed:   ffffffff,ffffffff
Cpus_allowed_list:      0-63
Mems_allowed:   00000000,00000001
Mems_allowed_list:      0
voluntary_ctxt_switches:        0
nonvoluntary_ctxt_switches:     0
```

查询文件句柄的当前使用情况：

```
$ cat /proc/sys/fs/file-nr
```

```
linux@ubuntu64-vm:/proc$ cat /proc/sys/fs/file-nr
6304    0       150474
```

file-nr 文件显示了三个参数：分配的文件句柄总数、当前使用的文件句柄数以及可以分配的最大文件句柄数。如果需要增大/proc/sys/fs/file-max 中的值，请确保正确设置 ulimit 。对于 2.4.20，通常将其设置为 unlimited 。使用 ulimit 命令来验证 ulimit 设置：

```
$ ulimit
```

```
linux@ubuntu64-vm:/proc$ ulimit
unlimited
```

通过 proc 文件系统修改内核中预定的一些变量.

修改整个系统中文件句柄的最大数量：

```
$ cat /proc/sys/fs/file-max
```

$ echo 65536 >/proc/sys/fs/file-max

$ cat /proc/sys/fs/file-max

```
root@ubuntu64-vm:/home/linux# cat /proc/sys/fs/file-max
150474
root@ubuntu64-vm:/home/linux# echo 65536 >/proc/sys/fs/file-max
root@ubuntu64-vm:/home/linux# cat /proc/sys/fs/file-max
65536
```

修改网络 TTL：

$ cat /proc/sys/net/ ipv4/ip_default_ttl

$ echo 128 >/proc/sys/net/ipv4/ip_default_ttl

$ cat /proc/sys/net/ ipv4/ip_default_ttl

```
root@ubuntu64-vm:/proc# cat /proc/sys/net/ipv4/ip_default_ttl
64
root@ubuntu64-vm:/proc# echo 128 >/proc/sys/net/ipv4/ip_default_ttl
root@ubuntu64-vm:/proc# cat /proc/sys/net/ipv4/ip_default_ttl
128
```

修改系统中最大进程数量：

$ cat /proc/sys/kernel/pid_max

$ echo 65536 >/proc/sys/kernel/pid_max

$cat /proc/sys/kernel/pid_max

```
root@ubuntu64-vm:/proc# cat /proc/sys/kernel/pid_max
32768
root@ubuntu64-vm:/proc# echo 65536 >/proc/sys/kernel/pid_max
root@ubuntu64-vm:/proc# cat /proc/sys/kernel/pid_max
65536
```

修改普通用户的最大 RTC 频率：

$ cat /proc/sys/dev/rtc/max-user-freq

$ echo 128 >/proc/sys/dev/rtc/max-user-freq

$ cat /proc/sys/dev/rtc/max-user-freq

```
root@ubuntu64-vm:/proc# cat /proc/sys/dev/hpet/max-user-freq
64
root@ubuntu64-vm:/proc# echo 128 >/proc/sys/dev/hpet/max-user-freq
root@ubuntu64-vm:/proc# cat /proc/sys/dev/hpet/max-user-freq
128
```

$ cat /proc/cpuinfo //查看 CPU 信息

```
root@ubuntu64-vm:/proc# cat /proc/cpuinfo
processor       : 0
vendor_id       : GenuineIntel
cpu family      : 6
model           : 158
model name      : Intel(R) Core(TM) i5-7500 CPU @ 3.40GHz
stepping        : 9
microcode       : 0x5e
cpu MHz         : 3408.080
cache size      : 6144 KB
fpu             : yes
fpu_exception   : yes
cpuid level     : 22
wp              : yes
flags           : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov
pat pse36 clflush dts mmx fxsr sse sse2 ss syscall nx pdpe1gb rdtscp lm constant
_tsc up arch_perfmon pebs bts nopl xtopology tsc_reliable nonstop_tsc aperfmperf
 pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt aes xsave a
vx f16c rdrand hypervisor lahf_lm 3dnowprefetch ida arat epb xsaveopt pln pts dt
s fsgsbase smep
bogomips        : 6816.16
clflush size    : 64
cache_alignment : 64
address sizes   : 40 bits physical, 48 bits virtual
power management:
```

$ cat /proc/interrupts //查看中断信息

```
root@ubuntu64-vm:/proc# cat /proc/interrupts
           CPU0
  0:         58   IO-APIC-edge      timer
  1:       8466   IO-APIC-edge      i8042
  4:       2520   IO-APIC-edge
  6:          3   IO-APIC-edge      floppy
  7:          0   IO-APIC-edge      parport0
  8:          1   IO-APIC-edge      rtc0
  9:          0   IO-APIC-fasteoi   acpi
 12:      42264   IO-APIC-edge      i8042
 14:          0   IO-APIC-edge      ata_piix
 15:       3462   IO-APIC-edge      ata_piix
 16:      36971   IO-APIC-fasteoi   snd_ens1371, vmwgfx
 17:       9089   IO-APIC-fasteoi   ehci_hcd:usb1, ioc0
 18:         79   IO-APIC-fasteoi   uhci_hcd:usb2
 19:      14840   IO-APIC-fasteoi   eth0
 40:          0   PCI-MSI-edge      PCIe PME, pciehp
 41:          0   PCI-MSI-edge      PCIe PME, pciehp
 42:          0   PCI-MSI-edge      PCIe PME, pciehp
 43:          0   PCI-MSI-edge      PCIe PME, pciehp
 44:          0   PCI-MSI-edge      PCIe PME, pciehp
 45:          0   PCI-MSI-edge      PCIe PME, pciehp
 46:          0   PCI-MSI-edge      PCIe PME, pciehp
 47:          0   PCI-MSI-edge      PCIe PME, pciehp
 48:          0   PCI-MSI-edge      PCIe PME, pciehp
 49:          0   PCI-MSI-edge      PCIe PME, pciehp
 50:          0   PCI-MSI-edge      PCIe PME, pciehp
 51:          0   PCI-MSI-edge      PCIe PME, pciehp
 52:          0   PCI-MSI-edge      PCIe PME, pciehp
 53:          0   PCI-MSI-edge      PCIe PME, pciehp
 54:          0   PCI-MSI-edge      PCIe PME, pciehp
 55:          0   PCI-MSI-edge      PCIe PME, pciehp
 56:          0   PCI-MSI-edge      PCIe PME, pciehp
 57:          0   PCI-MSI-edge      PCIe PME, pciehp
 58:          0   PCI-MSI-edge      PCIe PME, pciehp
```

$ cat /proc/ioports //设备 IO 端口

```
root@ubuntu64-vm:/proc# cat /proc/ioports
0000-0cf7 : PCI Bus 0000:00
  0000-001f : dma1
  0020-0021 : pic1
  0040-0043 : timer0
  0050-0053 : timer1
  0060-0060 : keyboard
  0064-0064 : keyboard
  0070-0071 : rtc0
  0080-008f : dma page reg
  00a0-00a1 : pic2
  00c0-00df : dma2
  00f0-00ff : fpu
  0170-0177 : 0000:00:07.1
    0170-0177 : ata_piix
  01f0-01f7 : 0000:00:07.1
    01f0-01f7 : ata_piix
  02f8-02ff : serial
  0376-0376 : 0000:00:07.1
    0376-0376 : ata_piix
  0378-037a : parport0
  03c0-03df : vga+
  03f2-03f2 : floppy
  03f4-03f5 : floppy
  03f6-03f6 : 0000:00:07.1
    03f6-03f6 : ata_piix
  03f7-03f7 : floppy
  03f8-03ff : serial
  0cf0-0cf1 : pnp 00:01
0cf8-0cff : PCI conf1
0d00-feff : PCI Bus 0000:00
```

$ cat /proc/meminfo //内存信息

```
root@ubuntu64-vm:/proc# cat /proc/meminfo
MemTotal:        1542776 kB
MemFree:          602688 kB
Buffers:           53644 kB
Cached:           376344 kB
SwapCached:            0 kB
Active:           491984 kB
Inactive:         322904 kB
Active(anon):     385580 kB
Inactive(anon):     5680 kB
Active(file):     106404 kB
Inactive(file):   317224 kB
Unevictable:           0 kB
Mlocked:               0 kB
SwapTotal:       2829308 kB
SwapFree:        2829308 kB
Dirty:                 0 kB
Writeback:             0 kB
AnonPages:        384916 kB
Mapped:           109444 kB
Shmem:              6364 kB
Slab:              59188 kB
SReclaimable:      35040 kB
```

$ cat /proc/partitions //所有设备的所有分区

```
root@ubuntu64-vm:/proc# cat /proc/partitions
major minor  #blocks  name

  11        0    1048575 sr0
   8        0   83886080 sda
   8        1   81053696 sda1
   8        2          1 sda2
   8        5    2829312 sda5
```

$ cat /proc/pci // PCI 设备的信息

```
root@ubuntu64-vm:/proc# cat devices
Character devices:
  1 mem
  4 /dev/vc/0
  4 tty
  4 ttyS
  5 /dev/tty
  5 /dev/console
  5 /dev/ptmx
  5 ttyprintk
  6 lp
  7 vcs
 10 misc
 13 input
 14 sound
 21 sg
 29 fb
 99 ppdev
108 ppp
116 alsa
128 ptm
136 pts
```

$ cat /proc/swaps //所有 Swap 分区的信息

```
root@ubuntu64-vm:/proc# cat /proc/swaps
Filename                                Type            Size    Used    Priority
/dev/sda5                               partition       2829308 0       -1
```

$ cat /proc/version // Linux 的版本号

```
root@ubuntu64-vm:/proc# cat version
Linux version 3.2.0-24-generic (buildd@yellow) (gcc version 4.6.3 (Ubuntu/Linaro 4.6.3-1ubuntu5)
) #37-Ubuntu SMP Wed Apr 25 08:43:22 UTC 2012
root@ubuntu64-vm:/proc#
```

3.3 使用 fork、exit 和 exec 系统调用编写多进程程序

命令行终端，执行：

$ cd ~/workdir/linux/application/9-process

$ mkdir fork

$ cd fork

```
linux@ubuntu64-vm:~/workdir/linux/application$ sudo mkdir 9-process
linux@ubuntu64-vm:~/workdir/linux/application$ cd 9-process
linux@ubuntu64-vm:~/workdir/linux/application/9-process$ sudo mkdir fork
linux@ubuntu64-vm:~/workdir/linux/application/9-process$ cd fork
```

将“华清远见-CORTEXA9 资料：\实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\09. Linux 系统 fork 等系统调用实验\实验代码”目录下的“93.c”拷贝到该目录下。

$ gcc 93.c –o fork

$ ./fork //执行观察结果

拷贝完成：

```
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ cd ..
linux@ubuntu64-vm:~/workdir/linux/application/9-process$ sudo cp /mnt/hgfs/share/实验代码/09
.\ Linux系统fork等系统调用实验/实验代码/93.c fork/ -a
linux@ubuntu64-vm:~/workdir/linux/application/9-process$ cd fork
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ ls
93.c
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ sudo gcc 93.c -o fork
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ ./fork
```

执行结果：

```
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ ./fork
the test content!
 fork test!
global=24 test=2 Parent,my PID is 3045
linux@ubuntu64-vm:~/workdir/linux/application/9-process/fork$ global=23 test=1 Child,my PID
is 3046
```

实验代码：

```c
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>

int global = 22;
char    buf[] = "the test content!\n";

int main(void)
{
    int test = 0, stat;
    pid_t pid;
    if(write(STDOUT_FILENO, buf, sizeof(buf)) != sizeof(buf))
    {
        perror("write error!");
    }
    printf(" fork test!\n");
    /* fork */
    pid = fork();   /*we should check the error*/
    if (pid == -1)
    {
        perror("fork");
        exit(0);
    }
    else if (pid == 0)
    {
        global++;
        test++;
        printf("global=%d test=%d Child,my PID is %d\n", global, test, getpid
```

```
    ());
            exit(0);
        }
        /*else be the parent*/
        global += 2;
        test += 2;
        printf("global=%d test=%d Parent,my PID is %d\n", global, test, getpid());
        exit(0);
    }
```

3.4 Linux 系统守护进程实验

运行 Ubuntu 系统，打开命令行终端。

执行命令

```
$ cd ~/workdir/linux/application/9-process
$ mkdir daemon
$ cd daemon
```

将“华清远见-CORTEXA9 资料：\实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\11. Linux 系统守护进程实验\实验代码”目录下的“init.c”和“test.c”拷贝到该目录下。

```
root@ubuntu64-vm:~/workdir/linux/application/9-process# cd daemon/
root@ubuntu64-vm:~/workdir/linux/application/9-process/daemon# ls
init.c  test  test.c
root@ubuntu64-vm:~/workdir/linux/application/9-process/daemon#
```

编译执行：

```
$ gcc init.c test.c –o test
$ ./test
```

```
root@ubuntu64-vm:~/workdir/linux/application/9-process/daemon# gcc init.c test.c -o test
root@ubuntu64-vm:~/workdir/linux/application/9-process/daemon# ./test
root@ubuntu64-vm:~/workdir/linux/application/9-process/daemon# ps -ef
```

执行结果如下图所示：

```
$ ps  -ef          //查看进程
```

```
root      3819     1  0 11:36 ?        00:00:00 ./test
root      3821  3537  0 11:36 pts/0    00:00:00 ps -ef
root@ubuntu64-vm:~/workdir/linux/application/9-process/daemon# cat /tmp/test.log
I'm here at Sat May 11 11:36:05 2024
nI'm here at Sat May 11 11:36:07 2024
nI'm here at Sat May 11 11:36:09 2024
nI'm here at Sat May 11 11:36:11 2024
nI'm here at Sat May 11 11:36:13 2024
nI'm here at Sat May 11 11:36:15 2024
nI'm here at Sat May 11 11:36:17 2024
nI'm here at Sat May 11 11:36:19 2024
nI'm here at Sat May 11 11:36:21 2024
nI'm here at Sat May 11 11:36:23 2024
nI'm here at Sat May 11 11:36:25 2024
nI'm here at Sat May 11 11:36:27 2024
nI'm here at Sat May 11 11:36:29 2024
nI'm here at Sat May 11 11:36:31 2024
nI'm here at Sat May 11 11:36:33 2024
nI'm here at Sat May 11 11:36:35 2024
nI'm here at Sat May 11 11:36:37 2024
nI'm here at Sat May 11 11:36:39 2024
nI'm here at Sat May 11 11:36:41 2024
nI'm here at Sat May 11 11:36:43 2024
nI'm here at Sat May 11 11:36:45 2024
nI'm here at Sat May 11 11:36:47 2024
nI'm here at Sat May 11 11:36:49 2024
nI'm here at Sat May 11 11:36:51 2024
nI'm here at Sat May 11 11:36:53 2024
nI'm here at Sat May 11 11:36:55 2024
nI'm here at Sat May 11 11:36:57 2024
nI'm here at Sat May 11 11:36:59 2024
nI'm here at Sat May 11 11:37:01 2024
nI'm here at Sat May 11 11:37:03 2024
nI'm here at Sat May 11 11:37:05 2024
nI'm here at Sat May 11 11:37:07 2024
nI'm here at Sat May 11 11:37:09 2024
nI'm here at Sat May 11 11:37:11 2024
nI'm here at Sat May 11 11:37:13 2024
nI'm here at Sat May 11 11:37:15 2024
nI'm here at Sat May 11 11:37:17 2024
nroot@ubuntu64-vm:~/workdir/linux/application/9-process/daemon#
```

实验代码：

init.c:

```c
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <stdlib.h>
void init_daemon(void)
{
    int pid;
    int i;

    if (pid = fork())
    {
        exit(0);
    }  //是父进程，结束父进程

    else if (pid < 0)
    {
        exit( -1 );
    }   //fork 失败，退出
```

```
    setsid();  //第一子进程成为新的会话组长和进程组长 并与控制终端分离

    if (pid = fork())
    {
        exit(0);
    } //是第一子进程，结束第一子进程
    else if (pid <  0)
    {
        exit(1);
    }   //fork 失败，退出

    //第二子进程不再是会话组长
    for(i = 0; i < getdtablesize(); ++i)
        //关闭打开的文件描述符
        close(i);
    chdir("/tmp");  //改变工作目录到 /tmp
    umask(0);  //重设文件创建掩模
    return;
}
```

test.c:

```
#include <stdio.h>
#include <time.h>
void init_daemon(void);  //守护进程初始化函数

int main()
{
    FILE *fp;
    time_t  t;
    init_daemon();  //初始化为 Daemon

    while(1)  //每隔 2 秒钟向 test.log 报告运行状态
    {
        sleep(2);  //睡眠 2 秒钟
        if((fp = fopen("test.log", "a")) != NULL)
        {
            t = time(0);
            fprintf(fp, "I'm here at %sn", asctime(localtime(&t)) );
            fclose(fp);
        }
    }
    return 0;
}
```

四、实验结论

在 Linux 多任务编程实验中，我深入学习了 Linux 系统中进程的概念和进程管理的基本原理。通过使用 ps 命令和/proc 文件系统查看系统中的进程信息，我掌握了如何获取进程 ID、状态及资源占用等关键数据。此外，通过编写守护进程程序，我理解了守护进程的编写步骤和特性，并且熟悉了使用 fork、exit 和 exec 系统调用实现多进程程序。这些实验不仅提升了我的多任务编程技能，还加深了我对进程间关系和通信方式的理解。

在实验过程中，我成功编写了多个程序，使用管道、消息队列和共享内存实现了简单的进程间通信。尤其是在守护进程实验中，通过编写和部署一个简单的守护进程，我体会到了守护进程在后台运行的特性及其应用场景。这次实验让我全面理解了 Linux 系统中多任务编程的基本方法，提升了我在 Linux 环境下进行复杂编程任务的能力，进一步巩固了我对进程管理和进程间通信的掌握。